Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Security Essentials

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

Security Properties

Security: what are we talking about?

Security is always about protecting something valuable (an asset) against threats by reducing risk to an acceptable level.

Think of it as a continuous balancing act between:

- What we want to protect (data, systems, reputation, money, lives)
- Who or what threatens it (attackers, accidents, insider threats, natural disasters)
- How much protection we can afford (cost, usability, performance trade-offs)

Security vs Safety

Security: protection of assets from both intentional harm (attacks, sabotage) and accidental harm (incidents, failures). Security is about managing risk, not eliminating it completely.

Safety: protection of people and the environment from unintentional harm (e.g., accidents, equipment failures, natural disasters).

Security vs Safety

Security: protection of assets from both intentional harm (attacks, sabotage) and accidental harm (incidents, failures). Security is about managing risk, not eliminating it completely.

Safety: protection of people and the environment from unintentional harm (e.g., accidents, equipment failures, natural disasters).

Key difference:

- Safety focuses on **accidents**.
- Security considers **malicious actions**.

Physical Security

Protection of physical assets (buildings, rooms, hardware, documents) against theft, damage, and unauthorized physical access.

Mechanisms include:

- Locks, fences, barriers
- Guards, security personnel
- Access badges, biometric scanners
- CCTV, motion detectors, alarms

Physical Security

Protection of physical assets (buildings, rooms, hardware, documents) against theft, damage, and unauthorized physical access.

Mechanisms include:

- Locks, fences, barriers
- Guards, security personnel
- Access badges, biometric scanners
- CCTV, motion detectors, alarms

> **NOTE:** Even the best cybersecurity is useless if someone can walk into your server room and steal the disks!

Computer Security

Protection of computing resources (hosts, operating systems, applications, data) from unauthorized access, misuse, disruption, and modification.

Focus areas:

- Access control (who can do what)
- Malware protection (viruses, trojans, ransomware)
- Secure software development
- Patch management and vulnerability remediation
- Data protection (encryption, backups)

Cybersecurity

Computer security plus the broader context of networks, connected systems, users, and cyber-enabled threats.

Expands beyond individual computers to include:

- Network security (firewalls, intrusion detection, VPNs)
- Cloud security and distributed systems
- IoT and embedded device security
- Human factors (social engineering, phishing, insider threats)
- Organizational policies and incident response
- Legal and regulatory compliance (GDPR, HIPAA, etc.)

Cybersecurity

Computer security plus the broader context of networks, connected systems, users, and cyber-enabled threats.

Expands beyond individual computers to include:

- Network security (firewalls, intrusion detection, VPNs)
- Cloud security and distributed systems
- IoT and embedded device security
- Human factors (social engineering, phishing, insider threats)
- Organizational policies and incident response
- Legal and regulatory compliance (GDPR, HIPAA, etc.)

Cybersecurity = technical + human + organizational aspects

The CIA Triad

The classical goals of cybersecurity:

The CIA Triad

The classical goals of cybersecurity:

Confidentiality: ensure that information is accessible only to those authorized to have access

- Prevents unauthorized disclosure of sensitive data
- Techniques: encryption, access control, authentication

The CIA Triad

The classical goals of cybersecurity:

Confidentiality: ensure that information is accessible only to those authorized to have access

- Prevents unauthorized disclosure of sensitive data
- Techniques: encryption, access control, authentication

Integrity: ensure that information and systems are accurate and have not been tampered with

- Prevents unauthorized or undetected modification
- Techniques: checksums, digital signatures, version control

The CIA Triad

The classical goals of cybersecurity:

Confidentiality: ensure that information is accessible only to those authorized to have access

- Prevents unauthorized disclosure of sensitive data
- Techniques: encryption, access control, authentication

Integrity: ensure that information and systems are accurate and have not been tampered with

- Prevents unauthorized or undetected modification
- Techniques: checksums, digital signatures, version control

Availability: ensure that authorized users have reliable and timely access to information and resources

- Prevents denial of service (intentional or accidental)
- Techniques: redundancy, backups, load balancing, DDoS mitigation

Extended Security Properties

The CIA triad is often extended with:

Extended Security Properties

The CIA triad is often extended with:

Authenticity: verify that an entity (user, system, message) is who/what it claims to be

- Example: digital certificates, multi-factor authentication

Extended Security Properties

The CIA triad is often extended with:

Authenticity: verify that an entity (user, system, message) is who/what it claims to be

- Example: digital certificates, multi-factor authentication

Accountability / **Auditability**: ensure that actions can be traced back to responsible entities

- Example: audit logs, forensic analysis, compliance reporting

Extended Security Properties

The CIA triad is often extended with:

Authenticity: verify that an entity (user, system, message) is who/what it claims to be

- Example: digital certificates, multi-factor authentication

Accountability / **Auditability**: ensure that actions can be traced back to responsible entities

- Example: audit logs, forensic analysis, compliance reporting

Non-repudiation: prevent a party from denying that they performed an action

- Example: digital signatures on contracts, blockchain transactions

Extended Security Properties

The CIA triad is often extended with:

Authenticity: verify that an entity (user, system, message) is who/what it claims to be

- Example: digital certificates, multi-factor authentication

Accountability / **Auditability**: ensure that actions can be traced back to responsible entities

- Example: audit logs, forensic analysis, compliance reporting

Non-repudiation: prevent a party from denying that they performed an action

- Example: digital signatures on contracts, blockchain transactions

Privacy: control over personal data and its collection, processing, and sharing

- Broader than confidentiality: includes consent, purpose limitation, data minimization
- Example: GDPR rights (access, rectification, erasure, portability)

CIA Triad: What Goes Wrong?

Confidentiality breach: someone learns a secret they should not know

- Example: An attacker steals a database containing credit card numbers
- Example: An employee reads confidential salary information
- Impact: Loss of competitive advantage, identity theft, privacy violations

CIA Triad: What Goes Wrong?

Confidentiality breach: someone learns a secret they should not know

- Example: An attacker steals a database containing credit card numbers
- Example: An employee reads confidential salary information
- Impact: Loss of competitive advantage, identity theft, privacy violations

Integrity breach: someone changes something they should not change

- Example: An attacker modifies a bank transfer from $100 to 100{,}000$
- Example: Malware alters system logs to hide traces of an intrusion
- Impact: Financial fraud, loss of trust, incorrect decisions

CIA Triad: What Goes Wrong?

Confidentiality breach: someone learns a secret they should not know

- Example: An attacker steals a database containing credit card numbers
- Example: An employee reads confidential salary information
- Impact: Loss of competitive advantage, identity theft, privacy violations

Integrity breach: someone changes something they should not change

- Example: An attacker modifies a bank transfer from $100 to 100{,}000$
- Example: Malware alters system logs to hide traces of an intrusion
- Impact: Financial fraud, loss of trust, incorrect decisions

Availability breach: authorized users cannot access services or data

- Example: Ransomware encrypts all company files
- Example: DDoS attack overwhelms a website
- Impact: Business disruption, lost revenue, reputational damage

Data

Data means Bits

When we communicate, we exchange data: text messages, images, audio, video, sensor readings, machine learning models, financial transactions...

Data means Bits

When we communicate, we exchange data: text messages, images, audio, video, sensor readings, machine learning models, financial transactions...

FUNDAMENTAL INSIGHT:
At the lowest level, ***all*** digital data is stored and transmitted as **bytes** (8-bit chunks):

- Bytes are just sequences of **bits** (binary digits: 0 or 1).
- A photo, a song, a database, an encrypted message—they're all ultimately just long strings of 0s and 1s.

Why Bits? Reliable Physical States

Physical devices naturally offer two stable, distinguishable states:

- Transistors: conducting vs non-conducting
- Voltage levels: high ($\sim$5V) vs low ($\sim$0V)
- Magnetic domains: north vs south
- Optical media: reflective vs non-reflective pits

Why Bits? Reliable Physical States

Physical devices naturally offer two stable, distinguishable states:

- Transistors: conducting vs non-conducting
- Voltage levels: high ($\sim$5V) vs low ($\sim$0V)
- Magnetic domains: north vs south
- Optical media: reflective vs non-reflective pits

Two-state encoding provides:

- Noise margins: easier to distinguish "definitely 0" from "definitely 1" even with interference
- Reliability: less susceptible to errors than multi-level encoding
- Simplicity: cheaper and faster circuits

Why Bits? Reliable Physical States

Physical devices naturally offer two stable, distinguishable states:

- Transistors: conducting vs non-conducting
- Voltage levels: high ($\sim$5V) vs low ($\sim$0V)
- Magnetic domains: north vs south
- Optical media: reflective vs non-reflective pits

Two-state encoding provides:

- Noise margins: easier to distinguish "definitely 0" from "definitely 1" even with interference
- Reliability: less susceptible to errors than multi-level encoding
- Simplicity: cheaper and faster circuits

> Imagine trying to distinguish 10 different voltage levels reliably!

Why Bits? Boolean Logic

Binary values map perfectly to Boolean algebra:

- $0 = \text{False}, 1 = \text{True}$
- Logical operations: AND, OR, NOT, XOR
- These operations can be implemented with simple electronic gates
- Each gate can be implemented with a few transistors

Why Bits? Boolean Logic

Binary values map perfectly to Boolean algebra:

- 0 = False, 1 = True
- Logical operations: AND, OR, NOT, XOR
- These operations can be implemented with simple electronic gates
- Each gate can be implemented with a few transistors

This enables computation:

- Combine gates → build circuits → perform arithmetic, comparisons, decisions
- Example: XOR gate is fundamental to encryption (we'll see this later!)
- Example: Addition of binary numbers uses AND/OR/XOR gates

Why Bits? Boolean Logic

Binary values map perfectly to Boolean algebra:

- 0 = False, 1 = True
- Logical operations: AND, OR, NOT, XOR
- These operations can be implemented with simple electronic gates
- Each gate can be implemented with a few transistors

This enables computation:

- Combine gates → build circuits → perform arithmetic, comparisons, decisions
- Example: XOR gate is fundamental to encryption (we'll see this later!)
- Example: Addition of binary numbers uses AND/OR/XOR gates

Binary also maps to arithmetic:

- Positional notation: $1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11_{10}$
- All mathematical operations can be performed in binary

Why Bits? Abstraction Layers

Bits enable a powerful hierarchy of abstractions:

Bits (0, 1)

↓

Bytes (8 bits = 256 possible values)

↓

Data types (integers, floats, characters, booleans)

↓

Data structures (arrays, objects, databases)

↓

Files and protocols (JPEG, MP4, HTTP, TLS)

↓

Applications (web browsers, email clients, streaming services)

Data transformations

Several approaches to transform bytes with fundamentally different purposes:

Data transformations

Several approaches to transform bytes with fundamentally different purposes:

Concept	Primary Goal	Security?
Encoding	Represent data using a different alphabet/format	✗ no
Serialization	Convert structured data into bytes	✗ no
Compression	Reduce size by removing redundancy	✗ no
Encryption	Protect confidentiality (hide content)	✓ yes

Encoding

Encoding changes representation without changing meaning:

- Purpose: Make data compatible with systems that expect certain formats
- Example: Email systems historically handled ASCII text, so attachments are Base64-encoded
- Example: URLs can't contain spaces, so `hello world` becomes `hello%20world`

Encoding

Encoding changes representation without changing meaning:

- Purpose: Make data compatible with systems that expect certain formats
- Example: Email systems historically handled ASCII text, so attachments are Base64-encoded
- Example: URLs can't contain spaces, so `hello world` becomes `hello%20world`

NO SECURITY: Anyone can decode it (no secret key needed)

Encoding

Encoding changes representation without changing meaning:

- Purpose: Make data compatible with systems that expect certain formats
- Example: Email systems historically handled ASCII text, so attachments are Base64-encoded
- Example: URLs can't contain spaces, so `hello world` becomes `hello%20world`

NO SECURITY: Anyone can decode it (no secret key needed)

Examples: UTF-8, Base64, URL-encoding, ASCII, Hex

Serialization

Serialization is an encoding process that converts in-memory data structures to bytes (and back)

- Purpose: Store objects in files or send them over networks
- Example: Python dictionary → JSON string → bytes

Serialization

Serialization is an encoding process that converts in-memory data structures to bytes (and back)

- Purpose: Store objects in files or send them over networks
- Example: Python dictionary → JSON string → bytes

Examples: JSON, XML, Protobuf, MessagePack, Pickle

Serialization

Serialization is an encoding process that converts in-memory data structures to bytes (and back)

- Purpose: Store objects in files or send them over networks
- Example: Python dictionary → JSON string → bytes

Examples: JSON, XML, Protobuf, MessagePack, Pickle

> **NOTE**: Transferring raw bytes back to in-memory data is said to be ***deserialization***.

Serialization

Serialization is an encoding process that converts in-memory data structures to bytes (and back)

- Purpose: Store objects in files or send them over networks
- Example: Python dictionary → JSON string → bytes

Examples: JSON, XML, Protobuf, MessagePack, Pickle

NOTE: Transferring raw bytes back to in-memory data is said to be ***deserialization***.

NO SECURITY: Serialized data is often human-readable (JSON, XML)

Compression

Compression reduces size by exploiting patterns and redundancy

Compression

Compression reduces size by exploiting patterns and redundancy

- ***Lossless***: Perfect reconstruction
  - Examples: gzip, zstd, bzip2, PNG, ZIP

Compression

Compression reduces size by exploiting patterns and redundancy

- ***Lossless***: Perfect reconstruction
  - Examples: gzip, zstd, bzip2, PNG, ZIP
- ***Lossy***: Approximate reconstruction, smaller size
  - Examples: JPEG, MP3, H.264, WebP

Compression

Compression reduces size by exploiting patterns and redundancy

- ***Lossless***: Perfect reconstruction
  - Examples: gzip, zstd, bzip2, PNG, ZIP
- ***Lossy***: Approximate reconstruction, smaller size
  - Examples: JPEG, MP3, H.264, WebP

NO SECURITY: Compressed data can be decompressed by anyone

Compression

Compression reduces size by exploiting patterns and redundancy

- ***Lossless***: Perfect reconstruction
  - Examples: gzip, zstd, bzip2, PNG, ZIP
- ***Lossy***: Approximate reconstruction, smaller size
  - Examples: JPEG, MP3, H.264, WebP

NO SECURITY: Compressed data can be decompressed by anyone

NOTE: It is often combined with encryption (compress first, then encrypt)

Encryption

Encryption transforms data so only authorized parties (with the key) can read it

Encryption

Encryption transforms data so only authorized parties (with the key) can read it

SECURITY: Without the key, ciphertext should look like random noise

Encryption

Encryption transforms data so only authorized parties (with the key) can read it

SECURITY: Without the key, ciphertext should look like random noise

It comes in two flavors:

- ***Symmetric***: same key for encryption and decryption
  - Examples: AES, ChaCha20
- ***Asymmetric***: different keys for encryption and decryption
  - Examples: RSA, Elliptic Curve Cryptography

Encoding != Encryption

Even if the data have been encoded and may not be readable, they are not encrypted and can (and will!) be easily decrypted.

EXAMPLE:

- the base64 encoding of `password` is `cGFzc3dvcmQxMjM=`
- the encoding does not require a secret
- the algorithm is deterministic and known
- ...hence, the encoding is reversible!

Combining Operations

You can (and often should) combine these operations:

Combining Operations

You can (and often should) combine these operations:

1. Serialize your data structure → bytes
2. Compress the bytes → smaller bytes (optional)
3. Encrypt the bytes → ciphertext (for confidentiality)
4. Encode the ciphertext as Base64 → text-safe format (if needed)

Combining Operations

You can (and often should) combine these operations:

1. Serialize your data structure → bytes
2. Compress the bytes → smaller bytes (optional)
3. Encrypt the bytes → ciphertext (for confidentiality)
4. Encode the ciphertext as Base64 → text-safe format (if needed)

EXAMPLE: HTTPS does compression → encryption → transmission

Base64

Base64 is an encoding scheme, not encryption and not compression.

Why do we need Base64?

Historically, many systems were designed to handle only *text* (printable ASCII characters):

- Email protocols (SMTP) were originally 7-bit ASCII only
- URLs have restrictions on which characters are allowed
- JSON and XML are text formats that can't directly contain binary data
- Configuration files and logs are often text-based

Why do we need Base64?

Historically, many systems were designed to handle only *text* (printable ASCII characters):

- Email protocols (SMTP) were originally 7-bit ASCII only
- URLs have restrictions on which characters are allowed
- JSON and XML are text formats that can't directly contain binary data
- Configuration files and logs are often text-based

SOLUTION: Base64 converts binary data into a restricted set of 64 printable ASCII characters:

- Letters: `A-Z`, `a-z` (52 characters)
- Digits: `0-9` (10 characters)
- Symbols: `+`, `/` (2 characters)
- Padding: `=` (used to align to 4-character blocks)

How It Works

1. Consider the input as sequence of bits
   - Example: `"Cat"` in the ASCII encoding is `01000011 01100001 01110100` (24 bits)

* See next slide for the actual table

How It Works

1. Consider the input as sequence of bits
   - Example: `"Cat"` in the ASCII encoding is `01000011 01100001 01110100` (24 bits)

2. Split the sequence of bits into chunks of 6 bits each
   - Example:`010000 110110 000101 110100` (4 chunks of 6 bits)

* See next slide for the actual table

How It Works

1. Consider the input as sequence of bits
   - Example: `"Cat"` in the ASCII encoding is `01000011 01100001 01110100` (24 bits)

2. Split the sequence of bits into chunks of 6 bits each
   - Example:`010000 110110 000101 110100` (4 chunks of 6 bits)

3. Convert each 6-bit value (0-63) to a Base64 character according to a predefined table*
   - `010000` (16) → `Q`
   - `110110` (54) → `2`
   - `000101` (5) → `F`
   - `110100` (52) → `0`
   - Result: `"Cat"` → `"Q2F0"`

* See next slide for the actual table

How It Works

1. Consider the input as sequence of bits
   - Example: `"Cat"` in the ASCII encoding is `01000011 01100001 01110100` (24 bits)

2. Split the sequence of bits into chunks of 6 bits each
   - Example:`010000 110110 000101 110100` (4 chunks of 6 bits)

3. Convert each 6-bit value (0-63) to a Base64 character according to a predefined table*
   - `010000` (16) → `Q`
   - `110110` (54) → `2`
   - `000101` (5) → `F`
   - `110100` (52) → `0`
   - Result: `"Cat"` → `"Q2F0"`

4. If input length is not a multiple of 3, use = as padding
   - Example: `"Ca"` → `"Q2E="`

LUISS

* See next slide for the actual table

Base64 Mapping Table

Index	Binary	Char	Index	Binary	Char	Index	Binary	Char	Index	Binary	Char
0	000000	A	16	010000	Q	32	100000	g	48	110000	w
1	000001	B	17	010001	R	33	100001	h	49	110001	x
2	000010	C	18	010010	S	34	100010	i	50	110010	y
3	000011	D	19	010011	T	35	100011	j	51	110011	z
4	000100	E	20	010100	U	36	100100	k	52	110100	0
5	000101	F	21	010101	V	37	100101	l	53	110101	1
6	000110	G	22	010110	W	38	100110	m	54	110110	2
7	000111	H	23	010111	X	39	100111	n	55	110111	3
8	001000	I	24	011000	Y	40	101000	o	56	111000	4
9	001001	J	25	011001	Z	41	101001	p	57	111001	5
10	001010	K	26	011010	a	42	101010	q	58	111010	6
11	001011	L	27	011011	b	43	101011	r	59	111011	7
12	001100	M	28	011100	c	44	101100	s	60	111100	8
13	001101	N	29	011101	d	45	101101	t	61	111101	9
14	001110	O	30	011110	e	46	101110	u	62	111110	+
15	001111	P	31	011111	f	47	101111	v	63	111111	/